

# System Overview

---

## Ads Safety Prep

---

### Overview

---

A multi-agent ads safety pipeline that classifies shopping advertisements as suspicious or safe, using a combination of price anomaly detection and LLM-based content analysis.

Built as a learning project simulating real-world content moderation systems, with evaluation metrics and design documentation included.

---

### Project Structure

---

```
ads-safety-prep/  
  data/  
    ads.csv ← input advertisement data  
  src/  
    main.py ← pipeline entry point: CLI arguments, logging, orchestration  
    loader.py ← load CSV data, validate required columns  
    writer.py ← write results to output CSV files  
    llm_client.py ← wrapper for LLM API calls  
  agents/  
    init.py  
    price_agent.py ← detects suspicious pricing  
    illegal_agent.py ← detects illegal or restricted content  
    judge_agent.py ← aggregates results from multiple agents  
    safety_agent.py ← optional combined safety logic (legacy / extension)  
  tests/  
    test_price_agent.py  
    test_illegal_agent.py  
    test_api.py  
  output/  
    completed_results.csv ← full analysis results (generated)  
    suspicious_results.csv ← flagged ads only (generated)  
  README.md
```

---

# Tech Stack

| Component       | Technology    |
|-----------------|---------------|
| Language        | Python 3.x    |
| LLM             | Gemini API    |
| Data processing | pandas        |
| Environment     | python-dotenv |

## Getting Started

### Prerequisites

- Python 3.10+
- Git
- A Gemini API key

### Installation

```
git clone https://github.com/your-username/ads-safety-prep.git
cd ads-safety-prep
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

### Setup

```
bash cp .env.example .env
```

## Add GEMINI\_API\_KEY in .env file

...

### Running the Pipeline

Default input and output paths:

```
python3 src/main.py
```

Custom input and output paths:

```
python3 src/main.py --input data/ads.csv --output output/
```

View all available options:

```
python3 src/main.py --help
```

---

## Documentation

| Document                          | Description                                         |
|-----------------------------------|-----------------------------------------------------|
| <a href="#">Design Doc</a>        | Architecture, design decisions, and detection logic |
| <a href="#">Evaluation Report</a> | Evaluation results and error analysis               |

---

## Design Highlights

1. **Modular architecture:** Each agent handles a single responsibility
2. **LLM integration:** Agents can leverage LLMs for flexible reasoning
3. **Structured output validation:** Ensures LLM responses conform to strict JSON schema
4. **Extensibility:** New agents (e.g., ScamAgent, PolicyAgent) can be added easily
5. **Separation of concerns:** Clear distinction between data loading, analysis, and output
6. **Evaluation-driven development:** System quality measured with Precision and Recall on a manually labeled dataset

---

## Limitations

- Requires a valid Gemini API key — the system cannot run offline
- Text-only: image content in ads is not analyzed
- Not designed for production or distributed environments
- LLM outputs are non-deterministic; results may vary across runs

---

## Future Improvements

1. Add retry + fallback mechanisms for LLM calls

2. Introduce JSON schema validation and auto-repair
  3. Implement caching to reduce API cost
  4. Add more agents (e.g., ScamAgent, ComplianceAgent)
  5. Build a feedback loop for continuous improvement
- 

## Summary

---

This project demonstrates the transition from a simple rule-based pipeline to a scalable multi-agent system for ads safety detection, combining deterministic logic with LLM-based reasoning.

---

# Design Document

---

## Design Doc: Mini Shopping Ads Safety System

---

**Author:** Yiyi Wang **Date:** 04/13/2026 **Status:** Draft

---

### 1. Overview

---

This system simulates a simplified Google Ads Safety pipeline. It adopts a multi-agent architecture to classify shopping ads as either *suspicious* or *safe*, based on price anomalies and textual content.

In addition, a LoRA fine-tuning experiment was conducted to evaluate whether task-specific fine-tuning can improve classification performance compared to prompt-only baselines.

---

### 2. Problem Statement

---

Ads safety is a critical problem for online platforms for two main reasons:

- **Regulatory compliance:** Platforms must prevent misleading, illegal, or restricted advertisements to comply with laws and policies.
- **User trust and experience:** Low-quality or fraudulent ads reduce platform credibility and can harm users.

This system focuses on a specific subproblem: **determining whether a shopping ad is suspicious based on its price and textual description.**

Specifically, the system aims to detect:

- Unreasonable or anomalous pricing (e.g., luxury goods sold at extremely low prices)
  - Illegal or restricted products
  - Exaggerated or misleading descriptions
-

### 3. Goals / Non-Goals

---

#### Goals

- Detect whether the advertised product price is reasonable
- Identify illegal or suspicious products based on text
- Improve classification accuracy through fine-tuning

#### Non-Goals

- Detect suspicious content in images (no vision model is used)
  - Provide real-time classification at production scale
- 

### 4. Design Decisions

---

#### Decision 1: Why use a multi-agent system instead of a single agent?

- **Option A:** A single large agent handles all classification logic
- **Option B:** Multiple specialized agents + a JudgeAgent for aggregation

We chose **Option B** for the following reasons:

- Better **separation of concerns**
- Easier **debugging and error attribution**
- Improved **modularity and extensibility**

#### Trade-off:

- Higher cost due to multiple API calls per ad
- 

#### Decision 2: Why is Recall prioritized over Precision?

In an ads safety system:

- A **false negative** (missing a suspicious ad) can directly harm users and damage platform trust
- A **false positive** (flagging a safe ad) is less severe and can be corrected via manual review

Therefore, we prioritize **high Recall**, even at the cost of lower Precision.

---

### Decision 3: Why use Gemini API instead of a local model?

Gemini API was selected for the following reasons:

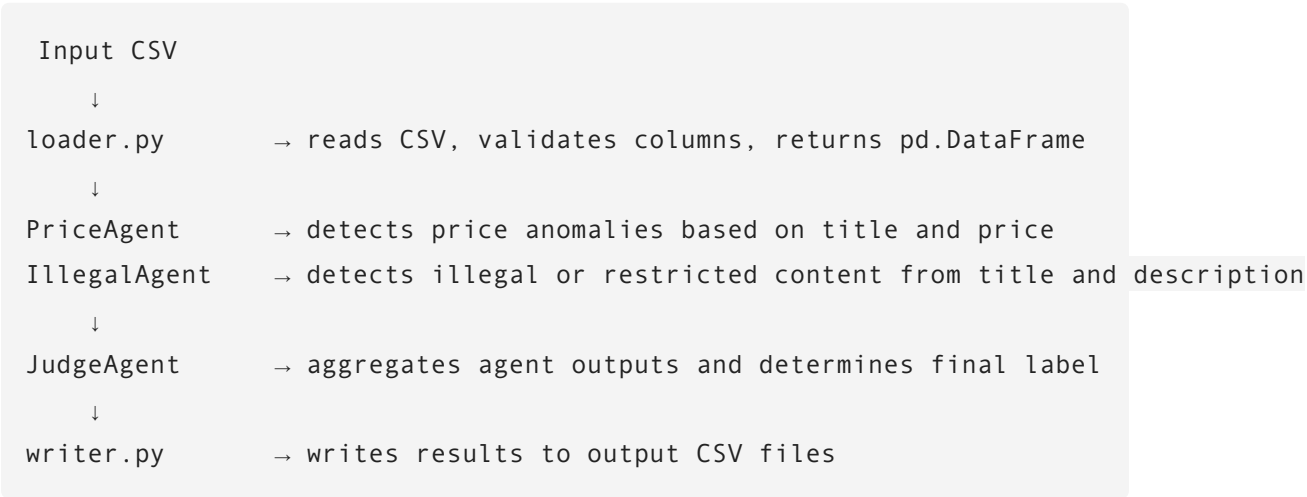
- No need for local GPU infrastructure
- Strong zero-shot reasoning capabilities
- Reduced need for large labeled datasets in early-stage prototyping

For a project of this scale, API cost is acceptable.

**Future direction:** In production, this would likely be replaced with a **fine-tuned lightweight model** (e.g., BERT-based classifier) to reduce cost and latency.

## 5. System Architecture

### 5.1 Pipeline Overview



The pipeline follows a modular multi-agent design. Each agent is responsible for a specific dimension of ad safety, enabling better extensibility, testability, and maintainability.

### 5.2 Agent Responsibilities

| Agent        | Input              | Output                                                                 | Responsibility                         |
|--------------|--------------------|------------------------------------------------------------------------|----------------------------------------|
| PriceAgent   | title, price       | { <code>"price_suspicious"</code> : bool, <code>"reason"</code> : str} | Detect whether the price is suspicious |
| IllegalAgent | title, description | { <code>"illegal_flag"</code> : bool, <code>"reason"</code> : str}     | Detect illegal or restricted content   |
| JudgeAgent   |                    |                                                                        |                                        |

| Agent | Input                           | Output                           | Responsibility                                       |
|-------|---------------------------------|----------------------------------|------------------------------------------------------|
|       | price_result,<br>illegal_result | {"label": str, "reason":<br>str} | Aggregate results and<br>determine final<br>decision |

### 5.3 Data Flow

| Stage              | Data Format                                          |
|--------------------|------------------------------------------------------|
| Input              | CSV file with columns: id, title, price, description |
| After Loader       | pd.DataFrame, validated and cleaned                  |
| After PriceAgent   | Dict: {"price_suspicious": bool, "reason": str}      |
| After IllegalAgent | Dict: {"illegal_flag": bool, "reason": str}          |
| After JudgeAgent   | Dict: {"label": "suspicious"/"safe", "reason": str}  |
| Output             | CSV file with original columns + label + reason      |
| ---                |                                                      |

## 6. Detection Logic

### Price-based Detection

1. Flag ads with unusually low prices
2. Combine price with brand signals for stronger detection

### Content-based Detection

1. Identify illegal or restricted keywords from ad text
2. Extendable to policy-based moderation rules

### Final Decision

1. Combine signals from multiple agents
2. Produce a unified label and explanation



## 7. Evaluation Plan

---

### Metrics

- **Precision:** Of all ads flagged as suspicious, what fraction are truly suspicious?
  - **Recall:** Of all truly suspicious ads, what fraction did the system successfully detect?
- 

### Why Recall is prioritized

In ads safety systems, missing harmful content is significantly more costly than over-flagging safe content. Therefore, Recall is the primary optimization target.

---

### Evaluation approach

We will manually label a dataset of N ads as ground truth, then compare system output against ground truth to compute precision and recall.

---

## 8. Limitations & Risks

---

1. **LLM reliability:** Outputs may be inconsistent or not strictly follow the expected JSON format
  2. **No learning loop:** The system does not improve from feedback
  3. **Limited policy coverage:** Only basic rules and prompts are implemented
  4. **Small-scale design:** Not suitable for distributed or production environments
  5. **No human-in-the-loop:** No moderation or review pipeline is implemented
- 

## 9. Future Work

---

To better align with industry practices, the system can be extended as follows:

- Introduce a **rule-based filtering layer** (e.g., regex patterns, keyword blacklists) as a first-stage filter
- Fine-tune **internal classification models** (e.g., BERT-based models) for each agent using domain-specific data
- Use these fine-tuned models as the primary classifiers for improved efficiency
- Route uncertain cases to **LLMs or human reviewers** for final decision-making

This hybrid approach balances **efficiency, scalability, and accuracy**, and is commonly used in production-level ads safety systems.

---

# Evaluation Report

---

## Evaluation Report: Mini Shopping Ads Safety System

---

**Date:** 04/14/2026 **Dataset:** data/labeled\_ads.csv **System version:** Multi-agent pipeline (PriceAgent + IllegalAgent + JudgeAgent)

---

### 1. Evaluation Setup

---

#### Dataset

- Total samples: 5
- Positive (suspicious) samples: 2
- Negative (safe) samples: 3
- Labeling method: manually labeled

#### Metrics used

- Precision
  - Recall
  - TP / FP / TN / FN
- 

### 2. Results

---

#### Confusion Matrix

|                    | Predicted: suspicious | Predicted: safe |
|--------------------|-----------------------|-----------------|
| Actual: suspicious | TP = 2                | FN = 0          |
| Actual: safe       | FP = 0                | TN = 3          |

Metric Summary

| Metric    | Value |
|-----------|-------|
| Precision | 1.000 |
| Recall    | 1.000 |
| TP        | 2     |
| FP        | 0     |
| TN        | 3     |
| FN        | 0     |

3. Error Analysis

False Negatives (漏报案例)

None in the final version.

False Positives (误报案例)

None in the final version.

In the initial version (v1), one false positive was observed:

- **id=4 | Perfume Gift Set | \$9.00**
- Predicted: suspicious / Actual: safe
- Root cause: PriceAgent flagged the price as suspicious without considering that generic, unbranded fragrance products are commonly priced in this range.

Root Cause Analysis (根本原因分析)

The initial false positive was caused by a prompt design issue in PriceAgent. The original prompt lacked guidance on how to handle product categories that are inherently low-priced. The model defaulted to treating any low price as suspicious, regardless of whether the product was a high-value branded item or a generic low-cost item.

The fix was to add explicit rules distinguishing between branded and unbranded products, with concrete examples for each case.

## 4. Iteration History

---

The system went through two prompt engineering iterations:

- **v1 (baseline):** PriceAgent prompt used a single example (brand-name smartphone) to illustrate suspicious pricing. This caused the model to over-generalize: any low price was treated as suspicious regardless of product category. Result: Precision=0.667, Recall=1.000
- **v2 (final):** PriceAgent prompt revised to include explicit rules distinguishing branded vs. unbranded products, with examples covering both suspicious and non-suspicious low-price cases. Result: Precision=1.000, Recall=1.000

---

## 5. Limitations of This Evaluation

---

1. **Sample size is very small.** Only 5 ads were evaluated. Results of 1.000/1.000 are not statistically meaningful at this scale and likely do not reflect real-world performance.
2. **No train/validation/test split.** The same dataset was used for both prompt development and final evaluation. This means the prompt may be overfit to these specific examples and could perform worse on unseen data.
3. **Label ambiguity.** At least one sample (Perfume Gift Set) was genuinely ambiguous. Even the human annotator was uncertain. Such samples introduce noise into the evaluation.
4. **Single evaluation run.** LLM outputs are non-deterministic. Running the pipeline once does not account for variance across runs.
5. **Limited category coverage.** The dataset does not cover many real-world ad categories such as electronics, clothing, food, or services.

---

## 6. Improvement Directions

---

1. **Expand the evaluation dataset** to at least 50-100 samples, covering more product categories and more edge cases.
2. **Separate validation and test sets** to avoid overfitting the prompt to the evaluation data.
3. **Run multiple evaluations** and report average Precision and Recall to account for LLM non-determinism. For example, run each sample 3–5 times and report mean  $\pm$  std to quantify output variance.

4. **Improve IllegalAgent coverage** — the current dataset does not stress-test illegal content detection sufficiently.
5. **Add a confidence signal** — rather than a binary suspicious/safe label, have the system output a risk level (low/medium/high) to support tiered human review workflows.